

System structure and external integrations (Sommerville notation)

June 30, 2026

This report describes the runtime structure of ScholarPath as a UML component diagram in the style of Sommerville. ScholarPath is a Clean Architecture monolith (**API** → **Application** → **Domain**; **Infrastructure** → **Application/Domain**) with a separate React single-page-application client and a set of external Azure / third-party services.

1	Notation	1
2	The component diagram	1
3	Component responsibilities	2
4	External integrations	2
5	Notable data flows	2

- A **component** is a box (here, a deployable assembly or external service). Components are grouped by their *deployment node* (Azure Static Web Apps, Azure App Service, external services).
- A **provided interface** is drawn as a lollipop ($\ominus$); a **required interface** as a socket ($\oslash$). In the source these are the application *ports* and the infrastructure *adapters* that provide them.
- A **dependency** (arrow) means one component uses another. Clean-Architecture dependencies point *inward*.

Figure 1: ScholarPath components, interfaces and external integrations.

3 Component responsibilities

Component	Provided interface (offers)	Requires
React SPA	the user-facing web app	API (REST), SignalR hubs
ScholarPath.API	HTTP/REST endpoints, JWT auth, Stripe webhook receiver	Application (MediatR)
SignalR Hubs	realtime push (chat, notifications, community)	Application
Hangfire Jobs	scheduled work (reminders, scholarship auto-close, no-show sweep, session-expiry)	Application
ScholarPath.Application	use cases (commands/queries) and ports	Domain
ScholarPath.Domain	entities, enums, domain events (the enterprise model)	nothing (innermost)
ScholarPath.Infrastructure	adapters that implement every port	Application, Domain, all external services

4 External integrations

Each external system is reached through one port and its production adapter (development uses `Local*/Stub*` fall-backs selected by configuration, e.g. `Ai__Provider`).

Port	Adapter	External system
<code>IApplicationDbContext</code>	<code>ApplicationDbContext</code> (EF Core)	Azure SQL
<code>IAiService</code>	<code>AzureOpenAiService</code>	Azure OpenAI (gpt-4o-mini)
<code>IEmbeddingService</code>	<code>AzureOpenAiEmbeddingService</code>	Azure OpenAI (text-embedding-3-small)
<code>IStripeService</code>	<code>StripeService</code>	Stripe (PaymentIntents, Connect, webhooks)
<code>IMeetingService</code>	<code>AzureCommunicationMeetingService</code>	Azure Communication Services (video + recording)
<code>IEmailService</code> (file storage)	<code>MailKitEmailService</code>	SMTP mail server
<code>IFileScanService</code>	<code>FileStorageService</code>	Azure Blob Storage
<code>IFileScanService</code>	<code>ClamAvFileScanService</code>	ClamAV daemon
<code>IFieldEncryptionService</code> (JWT signing)	<code>AesGcmFieldEncryptionService</code> (+ Key Vault)	Azure Key Vault
<code>IEventPublisher</code>	<code>KeyVaultJwtKeyProvider</code>	Azure Key Vault
<code>IPowerBiService</code>	<code>EventHubPublisher</code>	Azure Event Hub → Power BI
	<code>PowerBiService</code>	Power BI (embed + churn-risk reverse-ETL)

5 Notable data flows

- **Auth + API:** the React SPA calls the API over HTTPS with a JWT, and subscribes to the SignalR hubs over WebSocket for realtime updates.

- **Payments:** the Infrastructure layer creates / captures / refunds Stripe PaymentIntents; Stripe calls *back* into the API via a webhook, which is verified and recorded idempotently.
- **Analytics (reverse-ETL):** domain events are published to Azure Event Hub and consumed by Power BI; a daily Power BI dataflow writes churn-risk scores back into the **UserRiskFlags** table in Azure SQL.